



Introduction to Java

Lambdas and Streams



fausto.spoto@univr.it

A simple class that prints the first primes

```
public class Primes {  
    public void process(int howMany) {  
        for (long next = 2L; howMany > 0; howMany--, next++) {  
            while (!isPrime(next))  
                next++;  
  
            System.out.println(next);  
        }  
    }  
  
    private boolean isPrime(long n) {  
        for (long d = 2L; d * d <= n; d++)  
            if (n % d == 0L)  
                return false;  
  
        return true;  
    }  
}
```

What if we want to write the primes into a file?

```
public class Primes {  
    public void process(int howMany) {  
        try (PrintWriter pw = new PrintWriter("primes.txt")) {  
            for (long next = 2L; howMany > 0; howMany--, next++) {  
                while (!isPrime(next))  
                    next++;  
  
                System.out.println(next);  
                pw.println(next);  
            }  
        }  
        catch (IOException e) {  
            System.out.println("something went wrong!");  
        }  
    }  
  
    private boolean isPrime(long n) { ... }  
}
```

What if...

- ...we want to send the primes to the Internet
- ...we want to print the primes on the same line
- ...we want to print only a prime over two
- ...we want to show the primes on a graphical interface
- ...we want to add all the primes
- ...

Writing a class for each case is complex and becomes a maintenance problem

An abstract method wrapping the different implementation strategy might help, but we cannot use the same object with different strategies

The Hollywood principle: Don't call me, I'll call you

```
import java.util.function.Consumer;

public class Primes {
    public void process(int howMany, Consumer<Long> strategy) {
        for (long next = 2L; howMany > 0; howMany--, next++) {
            while (!isPrime(next))
                next++;

            strategy.accept(next);
        }
    }

    private boolean isPrime(long n) { ... }
}
```

```
public interface Consumer<T> {
    void accept(T);
}
```

Implementing Consumer with anonymous classes

```
public static void main(String[] args) {
    Primes primes = new Primes();
    primes.process(20, new Consumer<Long>() {
        public void accept(Long prime) {
            System.out.println(prime);
        }
    });
    try (PrintWriter pw = new PrintWriter("primes.txt")) {
        primes.process(20, new Consumer<Long>() {
            public void accept(Long prime) {
                pw.println(prime);
            }
        });
    }
    catch (IOException e) {
        System.out.println("something went wrong!");
    }
}
```

Boilerplate everywhere

```
public static void main(String[] args) {  
    Primes primes = new Primes();  
    primes.process(20, new Consumer<Long>() {  
        public void accept(Long prime) {  
            System.out.println(prime);  
        }  
    });  
    try (PrintWriter pw = new PrintWriter("primes.txt")) {  
        primes.process(20, new Consumer<Long>() {  
            public void accept(Long prime) {  
                pw.println(prime);  
            }  
    });  
}  
catch (IOException e) {  
    System.out.println("something went wrong!");  
}  
}
```

Enter the lambda

```
public static void main(String[] args) {  
    Primes primes = new Primes();  
    primes.process(20, prime -> System.out.println(prime));  
  
    try (PrintWriter pw = new PrintWriter("primes.txt")) {  
        primes.process(20, prime -> pw.println(prime));  
    }  
    catch (IOException e) {  
        System.out.println("something went wrong!");  
    }  
}
```


What is a lambda?

You can see a lambda as an implementation of an interface

```
Consumer<String> x = s -> System.out.println(s.substring(1));
```

The interface must have exactly one method (*functional* interface)

You can see a lambda as a pointer to a function

```
Consumer<Long> x = prime -> System.out.println(prime);  
primes.process(30, x);
```

Syntax of lambdas: Parameters

One parameter:

```
p -> p.translate(1, 1)
i -> new Point(i, i + 1)
```

Two parameters:

```
(x, y) -> x + y
(x, y) -> x.concat(y)
```

No parameters:

```
() -> 42
```

Syntax of lambdas: Argument types and return value

Types **can** be specified:

```
(Thread t) -> { t.start(); }  
(int i1, long i2) -> i1 + i2
```

Otherwise they are **inferred** from the context

A lambda

```
args -> expression
```

is equivalent to

```
args -> { return expression; }
```

Functional interfaces (SAM-types)

A **functional interface** is one with only a single abstract method, such as

```
public interface UnaryOperator<T> {  
    T apply(T t);  
}
```

Lambdas implement functional interfaces and can be used wherever one is expected: the following method of `java.util.List<E>`:

```
public void replaceAll(UnaryOperator<E> operator)
```

can be used as follows:

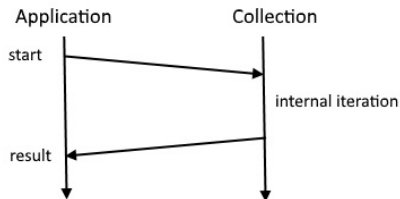
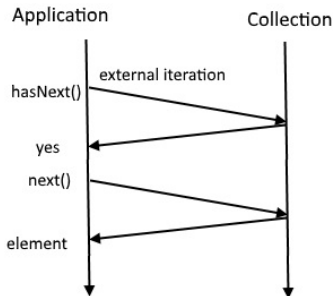
```
List<Integer> l = ...;  
l.replaceAll(i -> i + 1);  
l.replaceAll(i -> i * i);
```

Package java.util.function

Interface	Args.	Returns	Example
Consumer<T>	T	void	s -> System.out.print(s)
Predicate<T>	T	boolean	s -> s.isEmpty()
Supplier<T>	none	T	() -> new String()
Function<T,U>	T	U	s -> new Integer(s)

You are free to define your own functional interfaces, but use those above if you can, for standardization

External and internal iteration



Internal iteration is simpler, declarative, does not create any iterator and can be more efficient, by accessing the internal structure of the iterated collection, directly

From external to internal iteration

```
List<String> l = ... ;
```

External Iteration

We extract the elements of `l` and process each of them:

```
for (String s: l)
    process(s);
```

Internal Iteration

We provide the task that must be applied to each element of `l`

```
l.forEach(s -> process(s));
```

Examples of internal iteration

```
List<String> l = ... ;  
Set<String> copy = new HashSet<>();
```

Print all elements of the list

```
l.forEach(s -> System.out.println(s));
```

Add all elements to the copy

```
l.forEach(s -> copy.add(s));
```

For each element, create an instance of Processor

```
l.forEach(s -> new Processor(s));
```

For each element, call static method Util.digest

```
l.forEach(s -> Util.digest(s));
```


Examples of internal iteration

```
List<String> l = ... ;  
Set<String> copy = new HashSet<>();
```

Add to copy all elements of length at least 42

```
l.forEach(s ->  
    if (s.length() >= 42)  
        copy.add(s)  
);
```

For each element, if its length is at least 42, add it to copy. In any case, print all elements

```
l.forEach(s -> {  
    if (s.length() >= 42)  
        copy.add(s);  
  
    System.out.println(s);  
});
```

Method and constructor references

For compactness and efficiency, it is possible to replace

```
list.forEach(s -> System.out.println(s));  
list.forEach(s -> copy.add(s));  
list.forEach(s -> new Processor(s));  
list.forEach(s -> Util.digest(s));
```

with

```
list.forEach(System.out::println); // method reference  
list.forEach(copy::add); // method reference  
list.forEach(Processor::new); // constructor reference  
list.forEach(Util::digest); // method reference
```

Method and constructor references

Name	Syntax	Lambda Equivalent
Static	ClasName::staticMethod	(args) -> ClasName.staticMethod(args)
Bound Instance	expr::instMethod	(args) -> expr.instMethod(args)
Unbound Instance	ClsName::instMethod	(arg0,rest) -> arg0.instMethod(rest)
Constructor	ClsName::new	(args) -> new ClsName(args)
Array Creator	BasicType[]...[]::new 	(i1,...,in) -> new BasicType[i1]...[in]

Using method references

```
public static void main(String[] args) {  
    Primes primes = new Primes();  
    primes.process(20, prime -> System.out.println(prime));  
  
    try (PrintWriter pw = new PrintWriter("primes.txt")) {  
        primes.process(20, prime -> pw.println(prime));  
    }  
    catch (IOException e) {  
        System.out.println("something went wrong!");  
    }  
}
```

Using method references

```
public static void main(String[] args) {  
    Primes primes = new Primes();  
    primes.process(20, System.out::println);  
  
    try (PrintWriter pw = new PrintWriter("primes.txt")) {  
        primes.process(20, pw::println);  
    }  
    catch (IOException e) {  
        System.out.println("something went wrong!");  
    }  
}
```

Streams: `java.util.stream.Stream<T>`

A stream is a description of a lazy sequence of values

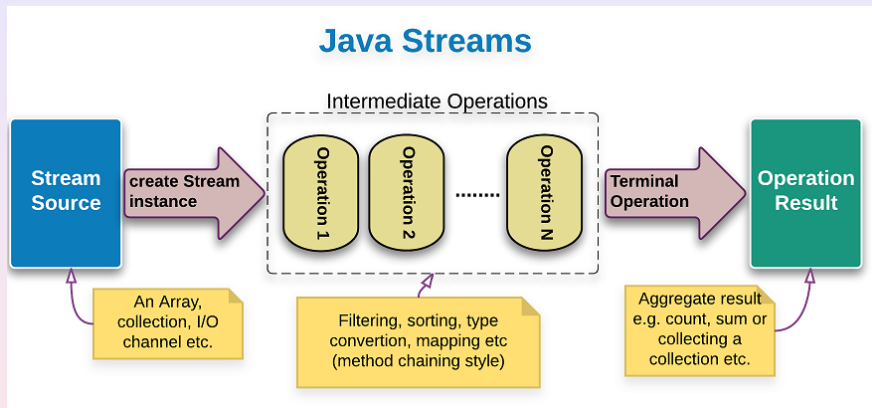
A stream is not a collection: it has no elements, it can even be infinite

Do not get confused with `FileInputStream` or similar: a stream is not necessarily for input/output

You can create streams from many sources:

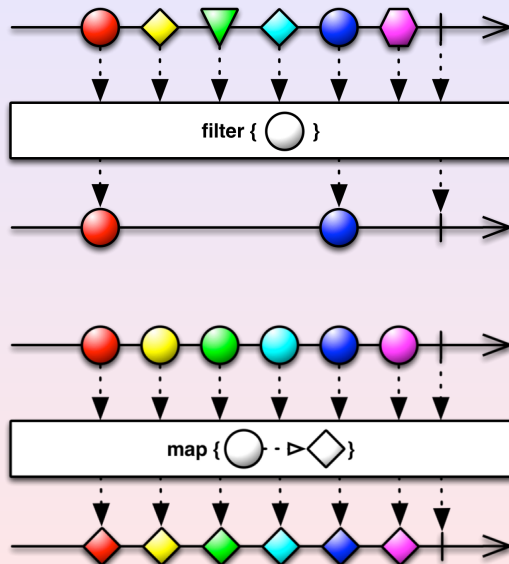
- from a collection `c`: `c.stream()`
- from elements or arrays: `Stream.of(a, b, c, ...)` or `Stream.of(array)`
- from a first element and next function: `Stream.iterate(first, next)`
- from an iterable:
`StreamSupport.stream(iterable.spliterator(), false)`
- from other data containers: for instance `BufferedReader.lines()` yields a stream of strings

Stream pipeline (create/filter/map/reduce)



Only the terminal operation runs the pipeline! (**lazy** streams)

Filter and map



Examples of stream pipelines

```
Collection<String> c = ... ;
```

Count how many elements of c start with "a"

```
long howMany = c.stream()  
                .filter(s -> s.startsWith("a"))  
                .count();
```

Print all elements of c, capitalized

```
c.stream()  
  .map(s -> s.toUpperCase())  
  .forEach(s -> System.out.println(s));
```

or equivalently with method references:

```
c.stream()  
  .map(String::toUpperCase)  
  .forEach(System.out::println);
```

Examples of stream pipelines

Print the elements of c of length at least 42, capitalized

```
c.stream()  
  .filter(s -> s.length() >= 42)  
  .map(s -> s.toUpperCase())  
  .forEach(s -> System.out.println(s));
```

or equivalently with method references:

```
c.stream()  
  .filter(s -> s.length() >= 42)  
  .map(String::toUpperCase)  
  .forEach(System.out::println);
```

Examples of stream pipelines

Find an element of `c` that starts with "a" and is at least 42 characters long

```
Optional<String> found = c.stream()
    .filter(s -> s.startsWith("a"))
    .filter(s -> s.length() >= 42)
    .findAny();

if (found.isPresent())
    System.out.println("here it is: " + found.get());
```

Examples of stream pipelines

Check if all elements of c are at least 42 characters long

```
boolean allAtLeast42 = c.stream()  
                        .allMatch(s -> s.length() >= 42);
```

Check if at least an element of c is at least 42 characters long

```
boolean oneAtLeast42 = c.stream()  
                      .anyMatch(s -> s.length() >= 42);
```

Check if no element of c is at least 42 characters long

```
boolean noneAtLeast42 = c.stream()  
                       .noneMatch(s -> s.length() >= 42);
```

Santa is a person who brings presents

```
public class Person {  
    private final String country;  
    public Person(String country)  
    public String getCountry() {  
        return country;  
    }  
}
```

```
public class Santa extends Person {  
    private final Set<String> presents;  
    public Santa(String country, Set<String> presents) {  
        super(country);  
        this.presents = presents;  
    }  
    public Stream<String> getPresents() {  
        return presents.stream();  
    }  
}
```

Examples of stream pipelines

```
Collection<Person> c = ...;
```

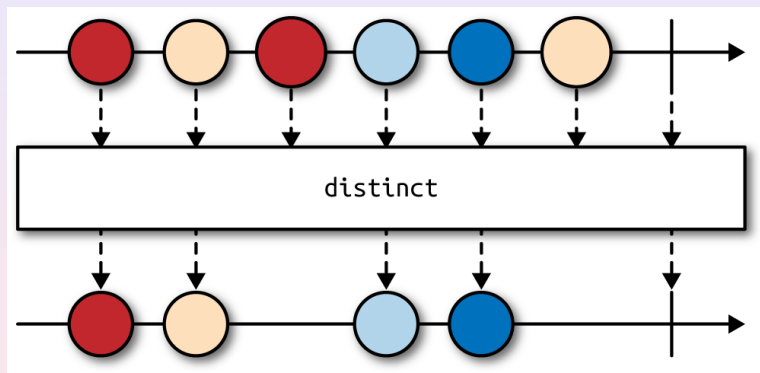
Count how many Santas are in c

```
long howMany =  
    c.stream()  
        .filter(person -> person instanceof Santa)  
        .count();
```

Count how many Santas from Finland are in c

```
long howMany =  
    c.stream()  
        .filter(person -> person instanceof Santa)  
        .filter(person -> person.getCountry().equals("Finland"))  
        .count();
```

Distinct



Changing type of the stream

Print the countries of all Santas in c, without repetitions

```
c.stream() // Stream<Person>
  .filter(person -> person instanceof Santa) // Stream<Person>
  .map(Person::getCountry) // Stream<String>
  .distinct() // Stream<String>
  .forEach(System.out::println);
```

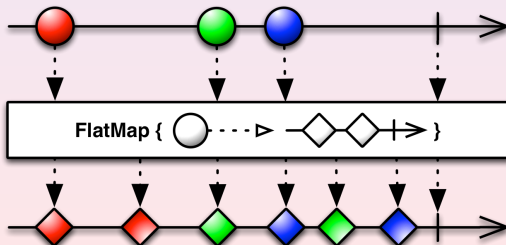
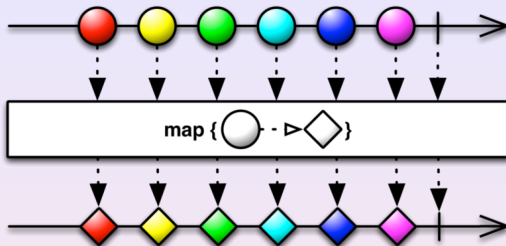
Check if at least a Santa in c lives in Finland

```
boolean isSantaInFinland =
  c.stream() // Stream<Person>
    .filter(person -> person instanceof Santa) // Stream<Person>
    .map(Person::getCountry) // Stream<String>
    .anyMatch(country -> "Finland".equals(country));
```

where the last line can be simplified into

```
....anyMatch("Finland"::equals);
```


Map vs flat map



Merging streams with flatMap

Print the names of all presents brought by the Santas in c, without repetitions and in ascending alphabetical order

```
c.stream() // Stream<Person>
  .filter(person -> person instanceof Santa) // Stream<Person>
  .map(person -> (Santa) person) // Stream<Santa>
  .flatMap(Santa::getPresents) // Stream<String>
  .distinct() // Stream<String>
  .sorted() // Stream<String>
  .foreach(System.out::println);
```

Parallel streams

Count how many presents bring the Santas in c

```
long howMany =  
    c.stream() // Stream<Person>  
      .filter(person -> person instanceof Santa) // Stream<Person>  
      .map(person -> (Santa) person) // Stream<Santa>  
      .map(Santa::getPresents) // Stream<Stream<String>>  
      .mapToLong(Stream::count) // LongStream  
      .sum();
```

Do the same in parallel

```
long howMany =  
    c.parallelStream() // Stream<Person>  
      .filter(person -> person instanceof Santa) // Stream<Person>  
      .map(person -> (Santa) person) // Stream<Santa>  
      .map(Santa::getPresents) // Stream<Stream<String>>  
      .mapToLong(Stream::count) // LongStream  
      .sum();
```

Streams of primitive values

LongStream vs. Stream<Long>

LongStream is a stream of primitive long values:

- it does not use boxing/unboxing $\Rightarrow$ more efficient
- it has specialized methods (`min()`, `max()`, `sum()`, ...)
- it has static methods for building ranges of values

Similarly, there are classes `IntStream` and `DoubleStream`

How to impress your friends

```
private boolean isPrime(long n) {  
    return LongStream.range(2L, (long) Math.sqrt(n) + 1L)  
        .noneMatch(d -> n % d == 0L);  
}
```

Scope and name capture

Lambdas define a **block scope** hence, as always in Java:

- local names shadow field names
- local names cannot shadow already existing external local names
- external names can be used inside a lambda (but only if they are **effectively final**)

Lambdas have no **object scope**, hence

- **this** refers to the same **this** as in its external scope, **not to the lambda itself**
- **super** refers to the superclass of the class where the lambda is defined, **not to the superinterface of the functional interface**

Effectively final variables

This code is correct, since variable `set` is effectively final

```
Set<String> set = ... // you could write final here
List<String> toAdd = ...
toAdd.forEach(set::add);
```

This code does not compile instead

```
Set<String> set = ...
List<String> toAdd = ...
set = new HashSet<>(set);
toAdd.forEach(set::add);
```

This code does not compile either

```
Set<String> set = ...
List<String> toAdd = ...
toAdd.forEach(set::add);
set = new HashSet<>(set);
```

Nightly read

